



FRIEDRICH-ALEXANDER-UNIVERSITÄT
ERLANGEN-NÜRNBERG (FAU)

NHR@FAU

Assignment 1: Code Execution

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

May 14, 2025

1. Pipelines (20 Credits)

We are given the following properties of a CPU pipeline:

- The floating-point **MULT** pipeline has a **latency** of 5 cycles.
- The pipeline has a **throughput** of 1 result per cycle (i.e., when fully utilized).
- We are required to compute how many independent **MULT** instructions are needed to reach **half of the maximum throughput**.

Pipeline Throughput Formula

According to Slide 29 of the lecture *Programming Techniques for Supercomputers (PTfS)*, the throughput of a pipelined functional unit is given by:

$$\text{Throughput} = \frac{N}{N + m - 1} \quad (1)$$

Where:

- N = Number of independent instructions
- m = Pipeline latency (in cycles)

We are given:

$$m = 5 \quad \text{and} \quad \text{Throughput} = 0.5 \text{ inst./cy}$$

Substituting into the equation:

$$0.5 = \frac{N}{N + 4} \quad (2)$$

Solving for N :

$$\begin{aligned} 0.5(N + 4) &= N \\ 0.5N + 2 &= N \\ 2 &= 0.5N \\ N &= 4 \end{aligned}$$

Answer

To achieve half of the maximum throughput (i.e., 0.5 results per cycle) on a 5-stage floating-point **MULT pipeline, at least 4 independent **MULT** instructions are required.**

This result matches the pipeline utilization model from the lecture, where performance improves as the number of independent instructions increases relative to the pipeline depth.

2. More Pipelines (30 Credits)

We are given a loop that updates each element of an array based on the value two positions before:

```
double a[...];
double s = 0.1;
for (int i = 2; i < N; ++i) {
    a[i] += s * a[i - 2];
}
```

Listing 1: Loop performing dependent computation

Instruction Analysis

Each iteration of the loop performs the following operations:

- Load $a[i - 2]$
- Multiply with scalar s
- Load $a[i]$
- Add the multiplication result to $a[i]$
- Store the final result back to $a[i]$

This results in a total of 5 instructions per iteration: 2 loads, 1 multiply, 1 add, and 1 store. Only the multiply and add are floating-point operations, contributing to the floating-point performance.

Hardware Constraints

The processor can issue one of each of the following per cycle:

- One floating-point add
- One floating-point multiply
- One load
- One store

In total, up to 4 instructions can be retired per cycle. Since there are 5 instructions per loop iteration, the instruction retirement bandwidth becomes the limiting factor.

Performance Calculation

Each iteration performs 2 floating-point operations (1 add and 1 multiply) and requires 5 instructions. With the processor capable of retiring 4 instructions per cycle, the performance can be estimated as:

$$\text{flops per cycle} = \frac{2}{5} \times 4 = \frac{8}{5} = 1.6 \quad (3)$$

Final Result

The expected floating-point performance of this loop is:

1.6 flops/cycle

This assumes optimal instruction scheduling and no stalls due to data access.

3. Logarithm in Depth (20 Credits)

In the benchmark code from Assignment 0, we approximate $\ln(2)$ by computing:

$$\int_0^1 \frac{1}{1+x} dx$$

using the midpoint rule:

```

sum = 0.0;
for (int i = 0; i < N; i++) {
    x = (i + 0.5) * delta_x;
    sum += 1.0 / (1.0 + x);
}

```

Here, the divide operation $1.0 / (1.0 + x)$ dominates the runtime. Since divide has a high latency and low throughput (e.g., 6 cycles latency, 0.25 instr./cycle), performance is limited if each iteration depends on the previous one through the summation variable `sum`.

Optimization Strategy

To hide divide latency and improve throughput, the compiler should apply loop unrolling and use multiple independent accumulators:

```

sum1 += 1.0 / (1.0 + x1);
sum2 += 1.0 / (1.0 + x2);
sum3 += 1.0 / (1.0 + x3);
sum4 += 1.0 / (1.0 + x4);

```

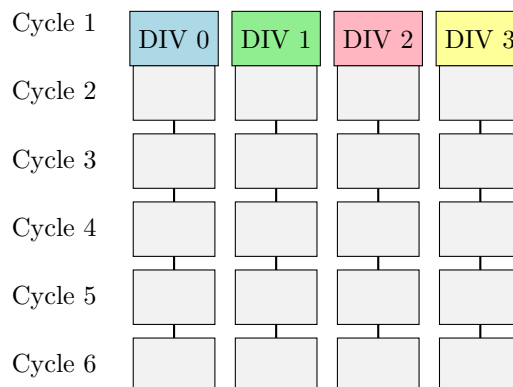
Each accumulator is independent, allowing the CPU to execute up to four divide instructions concurrently. After the loop, the final result is:

```

sum = sum1 + sum2 + sum3 + sum4;

```

Visualization



Conclusion

Unrolling the loop and using multiple accumulators removes the dependency bottleneck in `sum`. This allows the CPU to fully utilize the divide pipeline and significantly improves performance.

4. Memory Bandwidth (30 Credits)

(a) Bytes per cycle per core

Total memory bandwidth = 160 GB/s

Clock speed = 2.4 GHz = 2.4×10^9 cycles/s

Number of cores = 36

$$\text{Bytes/cycle/core} = \frac{160 \times 10^9 \text{ Bytes/s}}{2.4 \times 10^9 \text{ cycles/s} \times 36} = \frac{160}{86.4} \approx \boxed{1.85 \text{ Bytes/cycle/core}}$$

(b) Achievable ADD Performance with Memory Traffic

Each ADD instruction operates on 2 inputs and produces 1 output:

- Load 2 doubles: 16 Bytes
- Store 1 double: 8 Bytes
- Total = 24 Bytes per ADD

Let B = bytes per cycle per core = 1.85 (from part a)

$$\text{Max ADDs/cycle/core} = \frac{1.85 \text{ B/cycle}}{24 \text{ B/ADD}} \approx 0.077$$

Peak ADD performance = 1 ADD per cycle per core

$$\text{Achievable performance fraction} = \frac{0.077}{1} = \boxed{7.7\%}$$

Conclusion

Due to the high memory traffic per ADD operation, only about 7.7% of the peak ADD performance can be reached if all operands are streamed from and to main memory.